Advanced Free-Text Keystroke Dynamics: Synthesizing Biomechanical Rhythms and Ensemble Machine Learning for Continuous Authentication

The paradigm of cybersecurity and continuous user authentication has experienced a profound shift away from static, knowledge-based credentials toward dynamic, behavioral biometrics.[1] Passwords, personal identification numbers, and hardware tokens—while foundational to digital security—are inherently flawed; they verify the presence of a credential rather than the physical identity of the user. Consequently, they are susceptible to theft, sharing, and brute-force compromise.[1] To address these vulnerabilities, researchers have increasingly turned to keystroke dynamics, a highly sophisticated method of identity verification that continuously analyzes the unique, subconscious typing patterns of an individual.[1]

While traditional keystroke authentication models have primarily focused on fixed-text scenarios (e.g., analyzing the rhythm of a specific, predetermined password during a login event), the demand for seamless, continuous, and zero-trust security has driven the rapid development of free-text keystroke dynamics.[3] Free-text, or dynamic text, authentication allows the user to type freely without predetermined scripts, enabling systems to continuously verify identity in the background without interrupting the user's workflow.[3]

The core objective of an advanced free-text authentication system is a fundamental philosophical shift: the model must transition from analyzing *what* a person types (the linguistic content) to *how* they type it (the biomechanical execution). This involves extracting the inherent "rhythm" of the user, which is a direct, measurable manifestation of their underlying neuro-muscular control, complex finger movement trajectories, and physical muscle contraction speeds.[5] Because human beings generate keystroke timings based on complex, deeply ingrained habitual neurological pathways, these signals are highly individualized, extraordinarily difficult to consciously alter, and virtually impossible for an imposter to replicate or forge with precision.[2]

However, engineering a robust free-text authentication model that operates on rhythm and finger kinematics presents a unique constellation of challenges. Primary among these is the friction of user enrollment. To ensure high system adoption, enrolling a user must require minimal data input—ideally, capturing just a small, structured sample of character presses (such as the standard alphabet 'a-z' and numerals '0-9') while specifically ignoring structural or corrective keys (like space, backspace, and enter) to isolate the pure biomechanical targeting flow.[7] Transforming this minimal, sparse enrollment data into a comprehensive, highly accurate

continuous authentication model necessitates an advanced fusion of techniques. It requires state-of-the-art synthetic data generation to augment the sample size, rigorous spatial-temporal feature engineering to infer physical finger travel distance and muscle speed, and the deployment of complex ensemble machine learning architectures to predict user identity through democratic algorithmic voting.[9]

This exhaustive research report details the theoretical foundations, physiological mechanisms, spatial coordinate mappings, data synthesis strategies, and algorithmic architectures required to construct a state-of-the-art, ensemble-based free-text keystroke authentication system in Python.

The Physiological and Neuromotor Foundations of Typing Rhythms

To accurately model the "rhythm" of a typist, it is absolutely imperative to understand the physiological and biomechanical mechanisms that dictate keystroke timings. A single keystroke during touch-typing is not a simple binary or instantaneous event; rather, it is a rapid, goal-directed motion of the fingertip orchestrated by multiple motor neural centers in the brain and executed by the intricate extrinsic musculature of the hand and forearm.[5]

The Three-Burst Electromyographic (EMG) Pattern

Research utilizing fine-wire intramuscular electromyography (EMG) and high-speed, three-dimensional motion analysis has revealed that a single, standard keystroke is governed by a distinct, repeatable three-burst pattern of muscle activity.[5] This pattern underscores the complexity of what appears to be a simple motion.

1. **The Pre-Keystroke Extensor Burst:** The initial phase of a keystroke involves a burst of extensor muscle activity (specifically, the extensor digitorum communis), which lifts the finger in preparation for the downward strike toward the target key.[5] This preparatory lift is crucial for generating the necessary kinetic energy and spatial clearance.
2. **The Downward Flexor Burst:** The second phase is driven by a powerful burst of flexor activity (specifically, the flexor digitorum superficialis and flexor digitorum profundus). Interestingly, biomechanical data and force measurements suggest that this flexor contraction primarily serves to overcome the activation force and mechanical resistance of the keyboard keyswitch, rather than purely accelerating the finger downward through free space.[5] The collision with the end of the key's travel path is what abruptly halts the downward motion, transferring significant mechanical stress to the flexor tendons.[5] On a standard keyboard, the average peak reaction force during this collision is approximately 2.54 Newtons, representing roughly 10% of the muscle's maximum voluntary contraction (MVC).[14]
3. **The Post-Keystroke Extensor Burst:** The final phase features a second, precisely timed burst of extensor activity occurring exactly as the fingertip reaches the bottom of the key

travel. The timing of this third burst indicates that the primary role of the extensors at this stage is to rapidly remove the fingertip from the keyswitch, overcoming the key's upward return force and facilitating the transition to the next character in the typing sequence.[5]

Inferring Muscle Speed, Skill Level, and Neurological Status

The timing data passively captured by a computer system—specifically the microsecond latency between pressing and releasing keys—serves as a highly accurate digital proxy for these underlying muscle contractions and neural firing rates. Skilled typists demonstrate the ability to move their wrists and fingers faster and with less overall baseline muscle activity per keystroke, yet they exhibit highly localized, high-tension bursts in the finger flexors to achieve rapid key depression.[13] Unskilled typists, conversely, require significantly more time to switch between the flexion and extension phases due to higher cumulative loading on the extensor muscles, excessive immobilization of the wrist joint, and a lack of refined motor sequencing.[13]

By analyzing the micro-latencies between keystrokes, an advanced authentication model inherently captures the user's specific muscle contraction speed, tendon tension recovery time, and overarching neuro-motor coordination.[5] Furthermore, professional or highly skilled typists exhibit a high degree of multi-finger coordination, where the movement of one finger (e.g., the middle finger) anticipates and prepares the trajectory of an adjacent finger (e.g., the ring finger) due to interconnected tendon structures and advanced neural planning.[11] Capturing these overlapping temporal events is crucial for extracting the true, individualized rhythm of the user.

The biological basis of these rhythms is so profound that keystroke dynamics are increasingly utilized in the medical field as "Neurological Soft Signs" (NSS) to detect early sub-clinical motor abnormalities.[15] For instance, extensive clinical studies have demonstrated that individuals with Parkinson's Disease (PD) exhibit longer inter-key delays (flight times), shorter total distances of finger movement, and a phenomenon known as arrhythmokinesia—characterized by sudden hastening or freezing in the typing kinetics.[15] Similarly, typing speed and rhythm fluctuations have been highly correlated with cognitive decline, bipolar disorder manic phases, and processing speed deficits in multiple sclerosis.[15] Therefore, when a cybersecurity system measures a typing rhythm, it is fundamentally measuring a biological fingerprint dictated by the central nervous system.

Spatial-Temporal Feature Engineering: Extracting the Rhythm

In a continuous, free-text scenario, the sequence of words the user will type is entirely unpredictable. Therefore, the authentication system cannot rely on comparing the timing of a specific, known sequence (like a password). Instead, the system must rely on generalized timing features, aggregated statistical profiles, and spatial travel distances.[4] To capture the

rhythm effectively, we must engineer temporal features based on n-graphs (specifically digraphs and trigraphs) and physically map them to the geometric dimensions of the keyboard.[4]

During the critical enrollment phase, it is vital to isolate the pure biomechanical flow of alphanumeric targeting. Therefore, structural, corrective, or navigational keys—such as 'Space', 'Backspace', 'Enter', 'Shift', and arrow keys—must be explicitly ignored by the listener script.[3] Including these keys introduces high variance and cognitive noise (e.g., stopping to think about formatting or correcting a typo), which obscures the raw, subconscious muscle rhythm.[18]

Temporal Features: Dwell and Flight Times

The foundational metrics of keystroke dynamics are extracted from the raw timestamps of Key-Down (Press) and Key-Up (Release) events triggered at the operating system level.[19] From these discrete events, several core temporal features are derived:

- **Dwell Time (Hold Time):** The duration a key remains physically depressed. It is calculated as the time difference between the Key-Up and Key-Down events of the exact same key.[21] Physiologically, this metric correlates strongly with the user's finger flexor tension, the key switch activation force, and the latency of the third-burst extensor reaction time required to lift the finger.[5]
- **Flight Time (Inter-key Latency):** The duration between releasing one key and pressing the subsequent key.[21] Flight time is highly indicative of the user's cognitive processing speed, finger travel speed through the air, and their spatial familiarity with the specific keyboard layout.[23]
- **Press-to-Press (Keydown-Keydown) Time:** The total time elapsed between the initial depression of two consecutive keys. This is effectively the mathematical sum of the Dwell Time of the first key and the Flight Time to the second key.[17]

The Role of Digraphs and Trigraphs

To capture the rhythm of continuous typing across unpredictable text, individual keystrokes are logically grouped into overlapping sequences known as n-graphs.[17]

- **Digraphs:** These represent the timing dynamics between any two consecutive keystrokes (e.g., the temporal transition from 't' to 'h').[4]
- **Trigraphs:** These represent the timing dynamics spanning three consecutive keystrokes (e.g., the temporal flow from 't' to 'h' to 'e').[4]

In advanced free-text authentication, the system does not merely look at global averages. Instead, it computes specific transition matrices. By calculating the mean, median, standard deviation, and variance of specific digraph and trigraph latencies across a user's typing session, the system builds a highly discriminative, multidimensional biometric profile.[4] Research indicates that while Dwell Time is useful, Flight Time across specific digraphs yields vastly

superior authentication results, as it captures the unique travel paths of the individual's fingers.[24]

Kinematic Spatial Modeling: Finger Distance and Muscle Speed

To transition from a simple timing analysis to a true physiological model that understands *how* a person types, the temporal data must be fused with spatial geometric data. Different keys require vastly different physical travel distances, which heavily influences the expected Flight Time.[25] For instance, moving a finger from 'A' to 'S' is a completely different biomechanical operation than moving a finger from 'Q' to 'M'. By mapping the standard QWERTY keyboard to a two-dimensional Cartesian coordinate system, the algorithm can calculate the exact geometric distance the fingers must travel between any two keystrokes.[25]

QWERTY Coordinate Mapping

The standard QWERTY layout features staggered rows, which requires precise fractional coordinate mapping to accurately reflect the physical distances between key centers.[25] The spacing between keys (key pitch) is standardized at approximately 19.05 mm (0.75 inches) horizontally and vertically, but the rows are offset by quarter or half fractions.[27]

To build the spatial model, we assign coordinates (X, Y) to each alphanumeric key, treating the upper-left key ('Q' or '1') as near the origin, or conversely, using a standard mathematical quadrant mapping. The following table illustrates a standardized Cartesian mapping for a QWERTY layout, accounting for the row staggering offsets [25]:

Keyboard Row	**Characters**	**Coordinate Representation Format (X,Y)**	**Offset Characteristics**
Row 1 (Numbers)	1, 2, 3, 4, 5, 6, 7, 8, 9, 0	$Y = 3$; X increments by 1	Base X alignment
Row 2 (Top Row)	Q, W, E, R, T, Y, U, I, O, P	$Y = 2$; X increments by 1	Staggered $+0.25$ or $+0.5$ from Row 1

Row 3 (Home Row)	A, S, D, F, G, H, J, K, L	$Y = 1$; X increments by 1	Staggered $+0.25$ or $+0.5$ from Row 2
Row 4 (Bottom)	Z, X, C, V, B, N, M	$Y = 0$; X increments by 1	Staggered $+0.5$ or $+0.75$ from Row 3

(Note: In programmatic implementations, distances are often normalized to "key-units" rather than precise millimeters, where 1 unit equals the distance between two adjacent keys on the same row.[26])

Distance Formulas: Euclidean vs. Manhattan

Using these assigned coordinate values, the theoretical travel distance D between a starting key at (X_1, Y_1) and a destination key at (X_2, Y_2) can be calculated. The most direct measurement is the Euclidean distance (the straight-line hypotenuse), which is applicable when a single finger glides over the keyboard [26]:

$$D_{Euclidean} = \sqrt{(X_2 - X_1)^2 + (Y_2 - Y_1)^2}$$

Alternatively, the Manhattan distance (L1 norm) is frequently utilized to represent the grid-like, orthographic movement constraints of human fingers, particularly when multi-finger touch-typing is employed [25]:

$$D_{Manhattan} = |X_2 - X_1| + |Y_2 - Y_1|$$

Deriving Finger Muscle Speed

Once the physical distance between two keys in a digraph is established, the system unlocks its most powerful biometric feature: **Finger Muscle Speed (Velocity)**.[25] By dividing the computed geometric travel distance by the recorded Flight Time for that specific digraph, the system calculates the velocity at which the user's hand and fingers are moving through space:

$$Velocity_{Finger} = \frac{D_{Euclidean}}{Time_{Flight}}$$

This derived kinematic feature is incredibly difficult for an imposter to replicate.[5] An attacker might be able to type at the same overall words-per-minute speed, but they cannot organically

replicate the exact micro-velocities a genuine user exerts when moving their index finger from 'T' to 'Y' versus their ring finger from 'W' to 'S'. By integrating this spatial-velocity data, the model stops measuring raw time and begins measuring the physics of the user's musculature.[29]

Overcoming Data Scarcity: Advanced Synthetic Data Generation

A primary constraint in successfully deploying keystroke dynamics in real-world scenarios is the friction of user enrollment. Users vehemently abhor lengthy registration processes that require them to type pages of text just to calibrate a biometric system. The ideal scenario, as mandated by the system requirements, involves a minimal enrollment phase—such as asking the user to type the alphabet (a-z) and numerals (0-9) a few times, while actively ignoring spaces and backspaces.[7]

However, this sparse, highly constrained dataset is fundamentally inadequate for training complex, high-dimensional machine learning ensembles. Deep learning and ensemble algorithms require vast amounts of data to establish tight decision boundaries; feeding them only 36 characters of data leads to severe overfitting, high variance, and an unacceptably high False Rejection Rate (FRR).[9] To bridge the gap between minimal enrollment data and the vast data requirements of modern machine learning, sophisticated synthetic data generation (SDG) techniques must be employed.[9] Data augmentation transforms a tiny sample into a massive, robust training matrix.[32]

Statistical and Gaussian Perturbation

The most fundamental method for generating synthetic keystroke data involves statistical perturbation using Gaussian distributions.[10] By analyzing the minimal enrollment samples (e.g., typing 'a-z' three times), the system calculates the baseline mean (μ) and standard deviation (σ) for each character's Dwell Time and the Flight Time of the transitions.[34]

Using these parameters, the Python system can generate thousands of simulated, synthetic keystroke vectors by drawing from a normal probability density function [34]:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

By introducing tiny amounts of random variance (noise) within the established standard deviation, the system creates data points that perfectly mimic the natural, slight biological inconsistencies of human typing.[34] While computationally inexpensive and highly effective for small dimensions, pure Gaussian synthesis assumes that all keystroke timings are independent. In reality, keystrokes are highly co-dependent (e.g., typing one key faster often leads to typing the next key faster due to momentum), meaning pure Gaussian noise may fail to capture the

complex, overlapping temporal dependencies of rapid touch-typing.[32]

Synthetic Minority Over-sampling Technique (SMOTE)

In the context of continuous authentication, the system constantly compares the genuine user's data (which is a minority class, as there is only one genuine user) against background noise or a massive database of imposter data (the majority class).[9] To handle this severe class imbalance within the multi-dimensional feature space, SMOTE is employed.[9]

Unlike simple naive duplication (which causes overfitting), SMOTE synthesizes new, realistic data points by geometrically interpolating between existing minority instances.[31] For a given real keystroke feature vector (e.g., a specific instance of typing 'abc'), SMOTE identifies its k-nearest neighbors in the high-dimensional feature space. It then selects a random neighbor and generates a new synthetic feature vector at a random point along the line segment connecting the two actual points.[31] This technique effectively expands and smooths the mathematical decision boundaries for the genuine user, greatly enhancing the robustness of the downstream classification models against slight natural variations in the user's typing rhythm without introducing impossible biological metrics.[9]

Generative Adversarial Networks (KeyGAN)

To capture the absolute most intricate subtleties of natural typing and biological rhythms from a tiny sample, deep learning generative models such as Generative Adversarial Networks (GANs) represent the state of the art.[10] A specialized architecture, such as KeyGAN, is designed specifically for synthesizing highly complex keystroke timing features from small initial samples.[10]

The KeyGAN architecture consists of two neural networks locked in a zero-sum adversarial game:

1. **The Generator (G):** Takes a random noise vector as input and attempts to mathematically synthesize highly realistic embedding feature vectors (incorporating Dwell Time, Flight Time, and computed finger velocities) that seamlessly mimic the genuine user's enrollment data.[37]
2. **The Discriminator (D):** Evaluates the generated synthetic vectors against the real enrollment vectors, outputting a probability that the sample is genuine or fake.[37]

Through continuous backpropagation, the Generator learns the multi-dimensional, non-linear correlations between different keys. It learns, for instance, that if the user types 't' quickly, they likely type 'h' quickly as well, preserving the biological momentum.[37] Studies have demonstrated that synthetic data produced by KeyGAN exhibits extremely low Fréchet Distances (< 0.03) and minimal Kullback–Leibler Divergences (< 700) when compared to real

data distributions, indicating near-perfect replication.[10] By heavily supplementing the minimal a-z/0-9 enrollment data with thousands of GAN-generated samples, the training dataset is expanded exponentially, allowing complex machine learning models to train effectively without degrading classification performance on real-world test sets.[10]

Architecting the Ten-Algorithm Ensemble Voting Classifier

Keystroke dynamics data, especially in an unconstrained free-text context, is inherently noisy. Factors such as transient cognitive fatigue, emotional stress, ambient temperature, posture, and keyboard hardware variations can induce significant temporal fluctuations in a user's rhythm.[18] Relying on a single machine learning algorithm to authenticate identity in such a volatile data environment often leads to a brittle system that is highly susceptible to unacceptable False Acceptance Rates (FAR) or False Rejection Rates (FRR).[9]

To construct a "perfect" model capable of flawless, continuous identity verification, an ensemble learning approach is unequivocally mandated.[9] Ensemble learning combines the predictive power of multiple distinct, heterogeneous base estimators, leveraging their individual mathematical strengths while mutually mitigating their respective weaknesses and biases.[31] By deploying a diverse array of 10 different machine learning algorithms and aggregating their predictions through a soft-voting mechanism, the system achieves unprecedented accuracy, stability, and generalization across varying typing conditions.[38]

The Ten Core Classification Algorithms

The following 10 algorithms are specifically selected to form the ensemble due to their distinct approaches to handling high-dimensional, temporal-spatial data [1]:

Algorithm Name	Core Mechanism	Specific Advantage for Keystroke Dynamics
1. Random Forest (RF)	Bootstrap aggregating of multiple decision trees with random feature subsets.[31]	Highly resistant to overfitting on high-dimensional synthetic data; handles non-linear geometric distances easily.[9]
2. Extra Trees (ETC)	Extreme randomization of tree split points without optimizing for information	Drastically reduces variance; effectively smooths out noisy

	gain.[1]	anomalies caused by momentary typing hesitations.[1]
3. Gradient Boosting (GBC)	Sequential tree building where each tree minimizes the residual errors of the previous one.[1]	Excels at isolating subtle, complex patterns in digraph latencies; frequently returns top standalone accuracy.[1]
4. XGBoost	Highly optimized, scalable gradient boosting with advanced L1/L2 regularization.[9]	Handles sparse matrices brilliantly (crucial for rare trigraphs); fast enough for real-time continuous evaluation.[43]
5. Bagging Classifier	Meta-estimator that aggregates predictions from base models trained on random subsets.[9]	Reduces variance and prevents isolated typing errors from triggering a false rejection of the user.[9]
6. Support Vector Machine (SVM)	Finds optimal hyperplanes to maximize class margins using kernel tricks (RBF).[1]	Maps non-linear physiological features (muscle speed) into higher dimensions where imposter data is linearly separable.[1]
7. K-Nearest Neighbors (KNN)	Instance-based classification relying on distance metrics to the k nearest training points.[1]	Adapts well to gradual shifts in user rhythm; highly responsive to Euclidean/Manhattan distance calculations.[1]
8. Decision Trees (CART)	Generates simple, explicit binary rules based on feature thresholds.[1]	Provides high interpretability; strictly segments data based on hard physiological thresholds (e.g., minimum flight time).[1]
9. Naive Bayes (NB)	Probabilistic model applying Bayes' theorem with assumptions of feature	Extremely fast and lightweight; provides a baseline prior probability of

	independence.[1]	user identity based on historical rhythm distributions.[1]
10. Multi-Layer Perceptron (MLP)	Feedforward neural network with hidden layers and non-linear activation functions.[40]	Deep feature representation; acts as a universal function approximator decoding intricate neuro-muscular interactions.[40]

The Ensemble "Soft Voting" Mechanism

To seamlessly synthesize the outputs of these 10 disparate algorithms, a Soft Voting Classifier mechanism is implemented.[38] In a Hard Voting system, each algorithm casts a strict binary vote (Genuine or Imposter), and the majority wins. However, this discards valuable confidence data. Soft Voting, conversely, calculates the weighted average of the predicted probabilities across all 10 classifiers.[38]

For a given continuous free-text keystroke sample matrix X, each algorithm M_i outputs a confidence probability $P_i(X)$ that the sample belongs to the genuine user. The final prediction $\hat{y}$ is determined by the argmax of the sum of the weighted probabilities:

$$\hat{y} = \arg\max \sum_{i-1}^{10} w_i P_i(X)$$

By granting higher weights (w_i) to the models that historically perform optimally on the specific user's cross-validation data (for instance, giving higher precedence to XGBoost and GBC for a user with highly consistent flight times), the ensemble leverages democratic statistical rules.[38] This effectively nullifies false rejections caused by anomalous, single-algorithm failures, creating an exceptionally resilient and mathematically sound continuous authentication shield.[38]

System Implementation Blueprint in Python

Translating this complex theoretical and mathematical architecture into a functional, real-time Python application requires a robust, multi-threaded, event-driven design. The system must seamlessly intercept raw keystrokes at the hardware/OS level, rigorously ignore non-rhythmic keys, route the timing data through a spatial-temporal feature engineering pipeline, synthesize data for enrollment, and finally process the continuous stream via the 10-model ensemble to

output a real-time authentication score.[38]

1. The Data Acquisition and Key-Listening Layer

The absolute foundation of the Python system relies on the pynput library.[46] The pynput.keyboard module provides an asynchronous Listener class that hooks directly into the operating system's input stream, capturing hardware-level events with microsecond precision before they even reach the active application window.[46]

To isolate the pure biological rhythm and prevent structural formatting from skewing the data, the listener explicitly ignores keys such as space, backspace, enter, and shift, as mandated by the system requirements.[18]

Python

```
from pynput.keyboard import Listener, Key
import time

# Global buffer to store raw timestamps
keystroke_buffer =

# Define keys to ignore to capture pure alphanumeric rhythm
IGNORED_KEYS = [Key.space, Key.backspace, Key.enter, Key.shift, Key.shift_r, Key.tab]

def on_press(key):
    if key in IGNORED_KEYS:
        return # Skip structural keys

    try:
        char = key.char.lower() # Normalize to lowercase for consistency
        if char.isalnum(): # Ensure it is a-z or 0-9
            timestamp = time.time()
            keystroke_buffer.append({'key': char, 'event': 'press', 'time': timestamp})
    except AttributeError:
        pass # Handle special keys gracefully

def on_release(key):
    if key in IGNORED_KEYS:
        return
```

```
    try:
        char = key.char.lower()
        if char.isalnum():
            timestamp = time.time()
            keystroke_buffer.append({'key': char, 'event': 'release', 'time': timestamp})
    except AttributeError:
        pass

# Initialize and start the asynchronous listener thread
listener = Listener(on_press=on_press, on_release=on_release)
listener.start()
```

This module operates silently in the background, continually populating the keystroke_buffer with absolute Unix timestamps for both the precise depression and release of every valid alphanumeric key.[49]

2. The Feature Extraction and Geometric Distance Pipeline

A dedicated worker thread (or asynchronous loop) processes the raw keystroke_buffer in rolling chunks to extract the multidimensional features required by the machine learning models.

First, the module parses the buffer to calculate the base temporal metrics:

- **Dwell Time:** The mathematical difference between the release and press timestamps for a matching character event.[19]
- **Flight Time:** The difference between the press timestamp of the current key and the release timestamp of the immediately preceding key in the buffer.[19]

Next, the pipeline introduces the crucial spatial geometric mapping to infer finger movement.[26] A dictionary is defined in Python mapping every alphanumeric character to its specific (X, Y) coordinate on the staggered QWERTY matrix.[26] For every recognized digraph transition, the Euclidean distance is calculated:

Python

```
import math
import pandas as pd

# QWERTY Cartesian Coordinate Mapping (Row 1-4, staggered)
```

```
qwerty_coords = {
    '1': (0, 3), '2': (1, 3), '3': (2, 3), '4': (3, 3), '5': (4, 3), '6': (5, 3), '7': (6, 3), '8': (7, 3), '9': (8, 3), '0': (9, 3),
    'q': (0.25, 2), 'w': (1.25, 2), 'e': (2.25, 2), 'r': (3.25, 2), 't': (4.25, 2), 'y': (5.25, 2), 'u': (6.25, 2), 'i': (7.25, 2), 'o': (8.25, 2), 'p': (9.25, 2),
    'a': (0.50, 1), 's': (1.50, 1), 'd': (2.50, 1), 'f': (3.50, 1), 'g': (4.50, 1), 'h': (5.50, 1), 'j': (6.50, 1), 'k': (7.50, 1), 'l': (8.50, 1),
    'z': (0.75, 0), 'x': (1.75, 0), 'c': (2.75, 0), 'v': (3.75, 0), 'b': (4.75, 0), 'n': (5.75, 0), 'm': (6.75, 0)
}

def calculate_finger_distance(key1, key2):
    """Calculates Euclidean distance between two QWERTY keys."""
    x1, y1 = qwerty_coords.get(key1, (0,0))
    x2, y2 = qwerty_coords.get(key2, (0,0))
    # Using math.hypot for optimal Euclidean distance calculation
    return math.hypot(x2 - x1, y2 - y1)

def extract_features(buffer_chunk):
    """Transforms raw buffer into a feature matrix including muscle speed."""
    features =
    for i in range(len(buffer_chunk) - 1):
        # Simplification of logic for demonstration
        key1 = buffer_chunk[i]['key']
        key2 = buffer_chunk[i+1]['key']

        flight_time = buffer_chunk[i+1]['time'] - buffer_chunk[i]['time']
        distance = calculate_finger_distance(key1, key2)

        # Calculate derived muscle/finger velocity
        muscle_speed = distance / flight_time if flight_time > 0 else 0

        features.append({
            'digraph': f"{key1}{key2}",
            'flight_time': flight_time,
            'distance': distance,
            'muscle_speed': muscle_speed
        })
    return pd.DataFrame(features)
```

The raw data is subsequently transformed into a highly structured pandas.DataFrame encompassing columns for specific digraphs, Flight Times, geometric travel distances, and the derived muscle velocities.[52]

3. Dataset Synthesis and User Enrollment

During the initial enrollment phase, the system prompts the user to rapidly type the alphabet and numerical sequence (abcdefghijklmnopqrstuvwxyz0123456789) just a handful of times to minimize friction.[7]

Once this minimal feature matrix is gathered, the Python script utilizes the imbalanced-learn (imblearn) and numpy libraries to synthesize a massive, deep-learning-ready training set.[31]

Python

```
from imblearn.over_sampling import SMOTE
import numpy as np

def synthesize_dataset(X_real, y_real, target_samples=10000):
    """Expands the minimal enrollment data using SMOTE and Gaussian Noise."""
    # Step 1: Apply SMOTE to geometrically interpolate new samples
    smote = SMOTE(sampling_strategy='minority', k_neighbors=2)
    X_smote, y_smote = smote.fit_resample(X_real, y_real)

    # Step 2: Apply Gaussian Perturbation for natural biological variance
    noise_factor = 0.02 # 2% variance
    gaussian_noise = np.random.normal(0, noise_factor, X_smote.shape)
    X_synthetic = X_smote + (X_smote * gaussian_noise)

    return X_synthetic, y_smote
```

Using SMOTE, the script iterates over the sparse minority class and interpolates synthetic keystroke vectors.[31] Furthermore, numpy.random.normal() is utilized to augment the dataset slightly by generating distributions centered around the user's mean values, applying a variance calculated from their biological inconsistencies.[34] This synthesized dataset is then concatenated, formatted, and normalized using sklearn.preprocessing.StandardScaler to ensure optimal gradient descent convergence during model training.

4. Constructing and Training the Ensemble

The final stage of the Python pipeline is powered by scikit-learn and xgboost. The system instantiates all ten individual classifiers, configures their hyperparameters, and wraps them in a unified Voting architecture.[31]

Python

```python
from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier, GradientBoostingClassifier, BaggingClassifier, VotingClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neural_network import MLPClassifier
from xgboost import XGBClassifier

def build_and_train_ensemble(X_train, y_train):
    """Initializes 10 models and trains the Soft Voting Ensemble."""

    # 1. Initialize the 10 distinct machine learning models
    rf = RandomForestClassifier(n_estimators=100, max_depth=20)
    etc = ExtraTreesClassifier(n_estimators=100)
    gbc = GradientBoostingClassifier()
    xgb = XGBClassifier(use_label_encoder=False, eval_metric='logloss')
    bag = BaggingClassifier()
    svm = SVC(probability=True, kernel='rbf') # Probability=True required for soft voting
    knn = KNeighborsClassifier(n_neighbors=5, metric='manhattan')
    cart = DecisionTreeClassifier()
    nb = GaussianNB()
    mlp = MLPClassifier(hidden_layer_sizes=(128, 64), activation='relu', max_iter=500)

    # 2. Aggregate into a Soft Voting Classifier
    ensemble_model = VotingClassifier(
        estimators=[
            ('rf', rf), ('etc', etc), ('gbc', gbc), ('xgb', xgb),
            ('bag', bag), ('svm', svm), ('knn', knn), ('cart', cart),
            ('nb', nb), ('mlp', mlp)
        ],
        voting='soft', # Averages the probability outputs
        weights=[1.5, 1.2, 1.5, 2.0, 1.0, 1.0, 1.0, 0.8, 0.5, 1.5] # Favoring robust gradient/tree models
    )

    # 3. Train the entire ensemble on the augmented synthetic dataset
    ensemble_model.fit(X_train, y_train)
```

```
    return ensemble_model
```

In a live production environment, the continuous data stream buffered by pynput is processed, geometrically mapped into spatial-velocity matrices, and fed sequentially into ensemble_model.predict_proba().[38] If the rolling average of the probability score drops below a rigorously defined, high-security confidence threshold (e.g., 0.85), the system flags an anomaly. It silently signals the overarching security infrastructure that an unauthorized user has usurped the keyboard, successfully identifying the imposter based purely on the biomechanics of *how* they type, completely agnostic to *what* text is currently being typed.[38]

Performance Evaluation and System Limitations

When evaluating the efficacy of such an advanced continuous authentication system, standard accuracy metrics are insufficient. The model must be assessed using biometric-specific metrics, primarily the Equal Error Rate (EER)—the precise threshold where the False Acceptance Rate (FAR, the likelihood of an imposter being granted access) perfectly intersects with the False Rejection Rate (FRR, the likelihood of the genuine user being locked out).[3]

By synthesizing data and leveraging a 10-algorithm ensemble, tests on standardized benchmarks (such as the CMU Keystroke Dynamics dataset) frequently push the EER below 1.5%, with optimized models like Multi-Layer Perceptrons and XGBoost driving errors down to fractions of a percent.[3] However, limitations exist. Free-text authentication requires a "window" of keystrokes to make an accurate prediction (often between 50 and 150 characters).[3] An imposter taking control of an unlocked machine could theoretically execute a devastatingly short command (e.g., rm -rf /) before the statistical window has captured enough digraphs and spatial velocities to detect the anomalous rhythm.[3] Therefore, keystroke dynamics is most effective when deployed as part of a layered, multi-factor continuous security architecture.

Synthesis and Conclusions

Free-text keystroke dynamics represents an absolute pinnacle of behavioral biometrics, successfully shifting the cybersecurity paradigm from periodic knowledge verification to relentless, continuous physiological observation. By recognizing that typing is a deeply ingrained neuro-motor function—characterized by distinct extensor and flexor muscle bursts, physical finger travel geometries across the QWERTY plane, and subconscious kinetic pacing—we can construct an authentication model that fundamentally understands the biomechanics of how an individual interacts with a physical interface.[5]

The system architecture outlined in this report comprehensively addresses the traditional limitations of keystroke dynamics. The challenge of sparse enrollment data—a major hurdle in user adoption and model viability—is solved through the deployment of sophisticated synthetic data generation techniques. By utilizing SMOTE, Gaussian perturbation, and Generative

Adversarial Networks (KeyGAN), the system constructs robust, high-dimensional biometric profiles from minimal initial inputs, ensuring user convenience without sacrificing cryptographic security.[9]

Furthermore, by combining spatial calculations (Cartesian coordinate mappings) with temporal features (dwell and flight times), the model moves beyond simple clocks and extracts highly resilient physical indicators, such as inferred muscle speed and multi-finger transition velocities.[25]

Finally, the integration of a 10-algorithm soft-voting ensemble—encompassing advanced tree-based methods, high-dimensional SVM hyperplane mapping, non-linear neural networks, and probabilistic models—ensures unparalleled accuracy and robustness against natural variations in human behavior.[31] This complex amalgamation of hardware-level Python listeners, biomechanical feature engineering, and democratic machine learning results in a seamless, non-intrusive, and virtually impenetrable continuous authentication framework.

Works cited

1. Comparing Machine Learning Classifiers for Continuous Authentication on Mobile Devices by Keystroke Dynamics - Semantic Scholar, accessed March 21, 2026, https://pdfs.semanticscholar.org/ddd9/fe25b663edf31fe316918b1f461c758bca85.pdf
2. Keystroke dynamics - Wikipedia, accessed March 21, 2026, https://en.wikipedia.org/wiki/Keystroke_dynamics
3. Keystroke Dynamics: Concepts, Techniques, and Applications - arXiv.org, accessed March 21, 2026, https://arxiv.org/html/2303.04605v3
4. (PDF) Are Digraphs Good for Free-Text Keystroke Dynamics? - ResearchGate, accessed March 21, 2026, https://www.researchgate.net/publication/224716304_Are_Digraphs_Good_for_Free-Text_Keystroke_Dynamics
5. Control Strategies for Finger Movement During Touch-Typing. The Role of the Extrinsic Muscles During a Keystroke - PubMed, accessed March 21, 2026, https://pubmed.ncbi.nlm.nih.gov/9698184/
6. (PDF) The physiology of keystroke dynamics - ResearchGate, accessed March 21, 2026, https://www.researchgate.net/publication/253642686_The_physiology_of_keystroke_dynamics
7. Statistical Modeling of Keystroke Dynamics Samples For the Generation of Synthetic Datasets - ResearchGate, accessed March 21, 2026, https://www.researchgate.net/publication/332317758_Statistical_Modeling_of_Keystroke_Dynamics_Samples_For_the_Generation_of_Synthetic_Datasets
8. Keystroke Dynamics Authentication Using a Small Number of Samples - ResearchGate, accessed March 21, 2026, https://www.researchgate.net/publication/347168662_Keystroke_Dynamics_Authentication_Using_a_Small_Number_of_Samples

9. Enhancing Keystroke Dynamics Authentication with Ensemble Learning and Data Resampling Techniques - ResearchGate, accessed March 21, 2026, https://www.researchgate.net/publication/386001896_Enhancing_Keystroke_Dynamics_Authentication_with_Ensemble_Learning_and_Data_Resampling_Techniques
10. KeyGAN: Synthetic keystroke data generation in the context of digital phenotyping - PubMed, accessed March 21, 2026, https://pubmed.ncbi.nlm.nih.gov/39615234/
11. A Model of Multi-Finger Coordination in Keystroke Movement - MDPI, accessed March 21, 2026, https://www.mdpi.com/1424-8220/24/4/1221
12. Control strategies for finger movement during touch-typing. The role of the extrinsic muscles during a keystroke - ResearchGate, accessed March 21, 2026, https://www.researchgate.net/publication/13587454_Control_strategies_for_finger_movement_during_touch-typing_The_role_of_the_extrinsic_muscles_during_a_keystroke
13. Relationship between the kinematics of wrist/finger joint movements and muscle loading during typing in people with different skill levels - medRxiv.org, accessed March 21, 2026, https://www.medrxiv.org/content/10.1101/2021.06.09.21258367v1.full-text
14. (PDF) Keyboard Reaction Force and Finger Flexor Electromyograms during Computer Keyboard Work - ResearchGate, accessed March 21, 2026, https://www.researchgate.net/publication/14230581_Keyboard_Reaction_Force_and_Finger_Flexor_Electromyograms_during_Computer_Keyboard_Work
15. Diagnostic accuracy of keystroke dynamics as digital biomarkers for fine motor decline in neuropsychiatric disorders: a systematic review and meta-analysis - PMC, accessed March 21, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC9095860/
16. Smartphone-derived Virtual Keyboard Dynamics Coupled with Accelerometer Data as a Window into Understanding Brain Health - PMC, accessed March 21, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC10729731/
17. Keystroke Dynamics for User Authentication and Identification by using Typing Rhythm - International Journal of Computer Applications, accessed March 21, 2026, https://www.ijcaonline.org/archives/volume144/number9/patil-2016-ijca-910432.pdf
18. Keystroke Dynamics Patterns While Writing Positive and Negative Opinions - PMC, accessed March 21, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC8434638/
19. nileshprasad137/keystroke-dynamics-datagen - GitHub, accessed March 21, 2026, https://github.com/nileshprasad137/keystroke-dynamics-datagen
20. Keystroke Dynamics Analysis and Prediction — Part 1/2 (EDA) | by Kartik Shenoy - Medium, accessed March 21, 2026, https://medium.com/data-science/keystroke-dynamics-analysis-and-prediction-part-1-eda-3fe2d25bac04
21. Exploring Keystroke Dynamics: Enhancing Authentication Through Typing

Patterns - Atlantis Press, accessed March 21, 2026, https://www.atlantis-press.com/article/126017399.pdf
22. Keystroke Dynamics - Biometrics Solutions, accessed March 21, 2026, https://www.biometric-solutions.com/keystroke-dynamics.html
23. Keystroke Dynamics - FI MUNI, accessed March 21, 2026, https://www.fi.muni.cz/~xsvenda/docs/KeystrokeDynamics2001.pdf
24. Improved Feature Engineering for Free-Text Keystroke Dynamics - ResearchGate, accessed March 21, 2026, https://www.researchgate.net/publication/344404662_Improved_Feature_Engineering_for_Free-Text_Keystroke_Dynamics
25. 1320. Minimum Distance to Type a Word Using Two Fingers - In-Depth Explanation, accessed March 21, 2026, https://algo.monster/liteproblems/1320
26. Keyboard Layouts & Finger Distance Guide | PDF - Scribd, accessed March 21, 2026, https://www.scribd.com/document/931892930/Kbd-Distance
27. Distances between keys on a QWERTY keyboard - Code Golf Stack Exchange, accessed March 21, 2026, https://codegolf.stackexchange.com/questions/233618/distances-between-keys-on-a-qwerty-keyboard
28. Dynamic programming solution to the shortest distance two-finger typing problem, accessed March 21, 2026, https://stackoverflow.com/questions/61164254/dynamic-programming-solution-to-the-shortest-distance-two-finger-typing-problem
29. High-accuracy detection of early Parkinson's Disease using multiple characteristics of finger movement while typing | PLOS One - Research journals, accessed March 21, 2026, https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0188226
30. Differential effects of type of keyboard playing task and tempo on surface EMG amplitudes of forearm muscles - PMC, accessed March 21, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC4554952/
31. Enhancing Keystroke Dynamics Authentication with Ensemble Learning and Data Resampling Techniques - MDPI, accessed March 21, 2026, https://www.mdpi.com/2079-9292/13/22/4559
32. KeyGAN: Synthetic keystroke data generation in the context of digital phenotyping | Request PDF - ResearchGate, accessed March 21, 2026, https://www.researchgate.net/publication/386323676_KeyGAN_Synthetic_keystroke_data_generation_in_the_context_of_digital_phenotyping
33. Can Synthetic Data Allow for Smaller Sample Sizes in Chronic Urticaria Research? - PMC, accessed March 21, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC12329239/
34. Generating synthetic data with Gaussian distribution - Stack Overflow, accessed March 21, 2026, https://stackoverflow.com/questions/59060030/generating-synthetic-data-with-gaussian-distribution
35. Machine Learning-Based Analysis of Free-Text Keystroke Dynamics - arXiv, accessed March 21, 2026, https://arxiv.org/pdf/2107.07409

36. Synthetic Keystroke Dynamics Generation Using a Generative Adversarial Network GAN | Request PDF - ResearchGate, accessed March 21, 2026, https://www.researchgate.net/publication/399792038_Synthetic_Keystroke_Dynamics_Generation_Using_a_Generative_Adversarial_Network_GAN
37. KeyGAN: Synthetic Keystroke Data Generation in the Context of Digital Phenotyping - DigitalCommons@TMC, accessed March 21, 2026, https://digitalcommons.library.tmc.edu/cgi/viewcontent.cgi?article=6600&context=uthgsbs_docs
38. An ensemble learning based IDS using Voting rule: VEL-IDS - PMC - NIH, accessed March 21, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC10557513/
39. enhancing user-level security: performance analysis of machine learning algorithms for dynamic, accessed March 21, 2026, http://www.jatit.org/volumes/Vol101No13/24Vol101No13.pdf
40. Machine Learning Techniques for Biometric Authentication Based on Keystroke Dynamics, accessed March 21, 2026, https://epublications.vu.lt/object/elaba:229583288/229583288.pdf
41. Keystroke dynamics refers to the automated method of identifying or confirming the identity of an individual based on the manner and the rhythm of typing on a keyboard - GitHub, accessed March 21, 2026, https://github.com/rakshithca/KeyStroke-Dynamics
42. Comparing Machine Learning Classifiers for Continuous Authentication on Mobile Devices by Keystroke Dynamics - MDPI, accessed March 21, 2026, https://www.mdpi.com/2079-9292/10/14/1622
43. Efficient Convolutional Neural Network-Based Keystroke Dynamics for Boosting User Authentication - PMC, accessed March 21, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC10220835/
44. On the Machine Learning Based Business Process for User Identification Using Keystroke Datasets - WCSE, accessed March 21, 2026, https://www.wcse.org/WCSE_2024/039.pdf
45. User authentication through behavioral biometrics using multi- class classification algorithms - kth .diva, accessed March 21, 2026, https://kth.diva-portal.org/smash/get/diva2:1846194/FULLTEXT01.pdf
46. Python Keylogger Tutorial - 4 - Using Pynput Library - YouTube, accessed March 21, 2026, https://www.youtube.com/watch?v=U53f8TX4SFM
47. How to control your mouse and keyboard using the pynput library in Python - TutorialsPoint, accessed March 21, 2026, https://www.tutorialspoint.com/article/how-to-control-your-mouse-and-keyboard-using-the-pynput-library-in-python
48. Simple Introduction To A Keystroke Monitor In Python | by Ronak Sharma | Medium, accessed March 21, 2026, https://medium.com/@ronak.d.sharma111/simple-introduction-to-a-keystroke-monitor-in-python-89834596aae3
49. Key Listeners in python? - Stack Overflow, accessed March 21, 2026, https://stackoverflow.com/questions/11918999/key-listeners-in-python
50. I Built a Lie Detector with Python and a Keyboard — It Kinda Worked? | by Ryan -

Medium, accessed March 21, 2026, https://medium.com/@123rayyan789/i-built-a-lie-detector-with-python-and-a-keyboard-it-kinda-worked-aadf7535e49e
51. Python Keylogger Tutorial - 6 - Listening for keyboard strokes - YouTube, accessed March 21, 2026, https://www.youtube.com/watch?v=En3wHnBRMn4
52. Build an N-Gram Text Analyzer for SEO using Python, accessed March 21, 2026, https://importsem.com/build-an-n-gram-text-analyzer-for-seo-using-python/
53. User Verification based on Keystroke Dynamics: Python code | Machine Learning in Action, accessed March 21, 2026, https://appliedmachinelearning.wordpress.com/2017/07/26/user-verification-based-on-keystroke-dynamics-python-code/
54. Dataset of human-written and synthesized samples of keystroke dynamics features for free-text inputs - PubMed, accessed March 21, 2026, https://pubmed.ncbi.nlm.nih.gov/37122930/